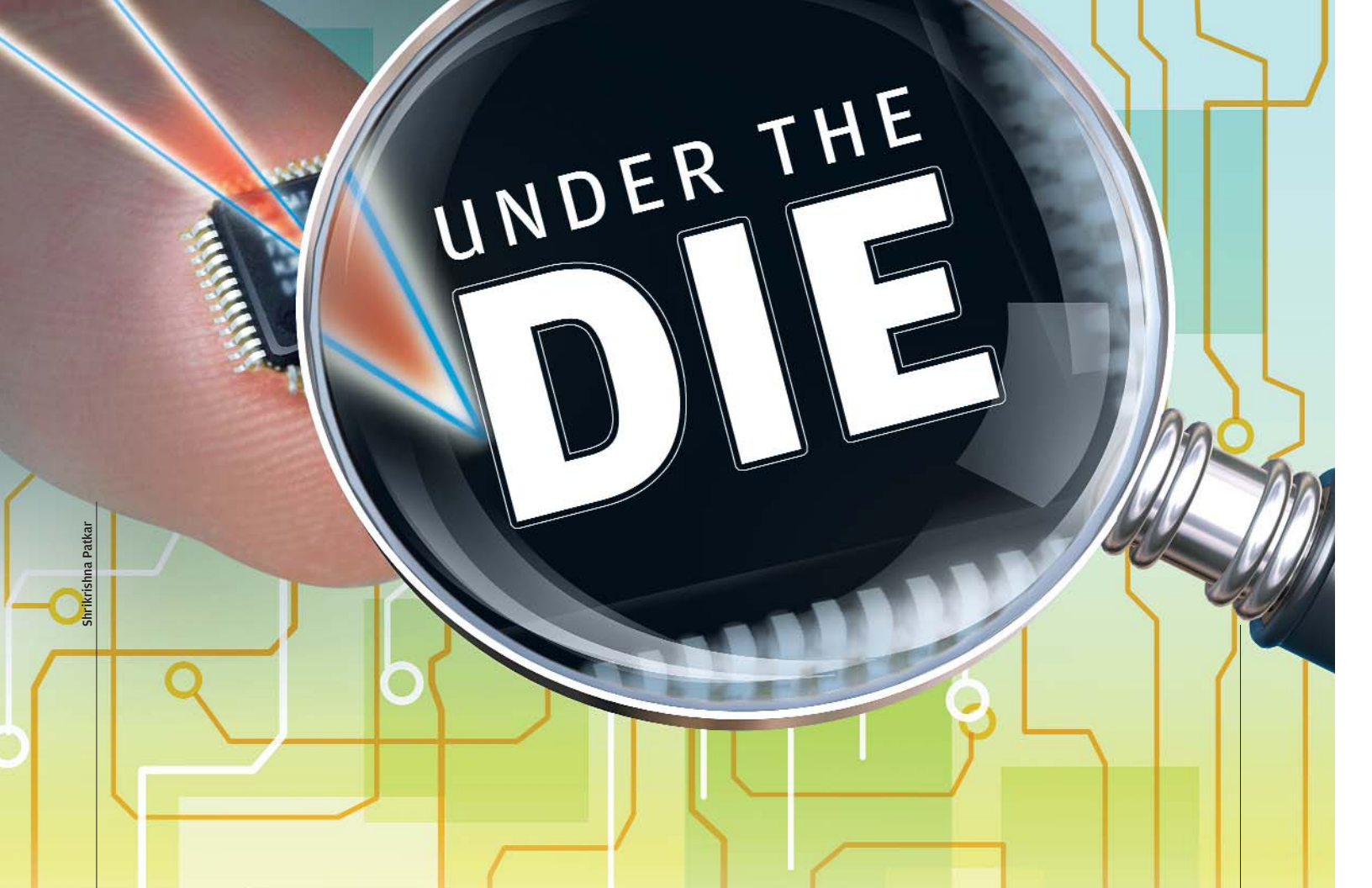




UNDER THE DIE

Shrikishna Patkar

How processors go from idea to your PC

Nimish Chandiramani

If you've ever handled a transistor—in your college lab, for instance—you've probably had a brain cell or two pop trying to imagine 291 million of them inside the diminutive package that is the Intel Core 2 Duo. And if it isn't enough to imagine just transistors on that little chip, try to imagine that those transistors are actually connected to each other in circuits, which ultimately power all our PCs. Hurts, doesn't it?

So how *do* you design circuitry when millions of transistors are involved? And how do you fit the lot on a chip that's barely the size of a fingernail?

Drawing Board

Before the actual design of your microchip commences, you first have to decide what it's going to do—and not just in vague terms like “Oh, it'll decode video files.” You'll have to draw up algorithms for how exactly the chip will decode video—which codec it'll use and so on.

The next step is dividing the chip's job into hardware and software—what tasks will be performed by the chip directly, and what you'll write drivers or software for. If you implement all the chip's functionality on the hardware itself, you'll have yourself a real cracking performer, but you won't be able to add to those functions later. For example, your mobile phone manufacturer could very well incorporate all the functions of your phone on to the same chip, but if they had to make even the simplest change to the phone—

like adding an FM radio function—they'd have to design and manufacture a whole new chip: an embarrassingly expensive proposition. Instead, they build a chip with all the essential features—making phone calls, for instance—and write software to implement the rest of the functions.

Once the hardware's role in the Big Picture is fixed, it's time to design.

Drawing Board, II

We talk about circuitry with millions of transistors, but the truth is that even in the early days of chip manufacture, designers never had to bother with individual transistors—the smallest units they had to care about were *logic gates*; this was a time when digital circuits weren't nearly as complex as today's. If you were to design a microprocessor at that minute level today, it would take years to just *finish* the design, let alone test and refine it. The solution? Don't bother with circuit diagrams at all—tell a computer what you want, and let it do the dirty work.

Today, designing hardware has become a matter of writing code—using a special “Hardware Descriptive Language” (HDL), you describe every function of the chip you're designing—right down to what the chip does in each clock cycle. That code, once interpreted (by a *logic synthesiser*), results in a circuit diagram—which can now be refined by hand, if necessary. Before your design is shipped off for manufacture, it'll need manual looking-over to see if the design can be optimised beyond what the logic synthesiser brought up.